

Libraries loaded successfully!

	delivery_id	delivery_partner	package_type	vehicle_type	delivery_mode
0	250.99	delhivery	automobile parts	bike	same day
1	250.99	xpressbees	cosmetics	ev van	express
2	250.99	shadowfax	groceries	truck	two days
3	250.99	dhl	electronics	ev van	same day
4	250.99	dhl	clothing	van	two days

Dataset Shape: (25000, 15)

```
['delivery_id', 'delivery_partner', 'package_type', 'vehicle_type', 'deli
```

```
<class 'pandas.core.frame.DataFrame'>
```

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	delivery_id	25000 non-null	float64
1	delivery_partner	25000 non-null	object
2	package_type	25000 non-null	object
3	vehicle_type	25000 non-null	object
4	delivery_mode	25000 non-null	object

```

5    region                25000 non-null object
6    weather_condition     25000 non-null object
7    distance_km           25000 non-null float64
8    package_weight_kg     25000 non-null float64
9    delivery_time_hours   25000 non-null object
10   expected_time_hours   25000 non-null object
11   delayed               25000 non-null object
12   delivery_status       25000 non-null object
13   delivery_rating       25000 non-null int64
14   delivery_cost         25000 non-null float64
dtypes: float64(4), int64(1), object(10)
memory usage: 2.9+ MB

```

```
# Check unique values in important columns
```

```

print("Delivery Time Samples:")
print(df['delivery_time_hours'].head(10))

print("\nExpected Time Samples:")
print(df['expected_time_hours'].head(10))

print("\nDelivery ID Samples:")
print(df['delivery_id'].head(10))

print("\nDuplicate Rows:", df.duplicated().sum())

```

```

Delivery Time Samples:
0    1970-01-01 00:00:00.000000008
1    1970-01-01 00:00:00.000000002
2    1970-01-01 00:00:00.000000010
3    1970-01-01 00:00:00.000000006
4    1970-01-01 00:00:00.000000009
5    1970-01-01 00:00:00.000000004
6    1970-01-01 00:00:00.000000006
7    1970-01-01 00:00:00.000000004
8    1970-01-01 00:00:00.000000005
9    1970-01-01 00:00:00.000000003
Name: delivery_time_hours, dtype: object

```

```

Expected Time Samples:
0    1970-01-01 00:00:00.000000008
1    1970-01-01 00:00:00.000000003
2    1970-01-01 00:00:00.000000016
3    1970-01-01 00:00:00.000000008
4    1970-01-01 00:00:00.000000016
5    1970-01-01 00:00:00.000000002
6    1970-01-01 00:00:00.000000008
7    1970-01-01 00:00:00.000000008
8    1970-01-01 00:00:00.000000008
9    1970-01-01 00:00:00.000000008
Name: expected_time_hours, dtype: object

```

```

Delivery ID Samples:
0    250.99
1    250.99

```

```
2    250.99
3    250.99
4    250.99
5    250.99
6    250.99
7    250.99
8    250.99
9    250.99
Name: delivery_id, dtype: float64
```

Duplicate Rows: 0

```
# Convert delivery and expected time into numeric hours

df['delivery_time_hours'] = pd.to_datetime(
    df['delivery_time_hours']
).dt.nanosecond

df['expected_time_hours'] = pd.to_datetime(
    df['expected_time_hours']
).dt.nanosecond

# Fix delivery_id to integer
df['delivery_id'] = df['delivery_id'].round().astype(int)

print("Data cleaning completed!")
print(df[['delivery_id', 'delivery_time_hours', 'expected_time_hours']].head(10))
```

```
Data cleaning completed!
   delivery_id  delivery_time_hours  expected_time_hours
0           251                   8                    8
1           251                   2                    3
2           251                  10                   16
3           251                   6                    8
4           251                   9                   16
5           251                   4                    2
6           251                   6                    8
7           251                   4                    8
8           251                   5                    8
9           251                   3                    8
```

```
# Fix suspicious delivery IDs
df['delivery_id'] = df['delivery_id'].round().astype(int)

print(df['delivery_id'].head(10))
print("Unique Delivery IDs:", df['delivery_id'].nunique())
```

```
0    251
1    251
2    251
3    251
4    251
5    251
6    251
7    251
```

```
8    251
9    251
Name: delivery_id, dtype: int64
Unique Delivery IDs: 24500
```

```
# Check missing values in every column
```

```
missing_values = df.isnull().sum()
```

```
print(missing_values)
```

```
delivery_id          0
delivery_partner      0
package_type         0
vehicle_type         0
delivery_mode        0
region              0
weather_condition    0
distance_km          0
package_weight_kg    0
delivery_time_hours  0
expected_time_hours  0
delayed              0
delivery_status      0
delivery_rating      0
delivery_cost        0
dtype: int64
```

```
# Check duplicate rows
```

```
duplicates = df.duplicated().sum()
```

```
print("Duplicate rows:", duplicates)
```

```
Duplicate rows: 0
```

```
# Basic statistics of numerical columns
```

```
print(df.describe())
```

	delivery_id	distance_km	package_weight_kg	delivery_time_hours
count	25000.000000	25000.000000	25000.000000	25000.000000
mean	12500.500000	150.390436	25.145898	6.248040
std	7212.731974	86.409745	14.368663	3.140935
min	251.000000	3.600000	0.670000	0.000000
25%	6250.750000	75.900000	12.680000	4.000000
50%	12500.500000	151.000000	25.145000	6.000000
75%	18750.250000	224.900000	37.660000	8.000000
max	24750.000000	297.100000	49.520000	19.000000

	expected_time_hours	delivery_rating	delivery_cost
count	25000.000000	25000.000000	25000.000000
mean	13.107680	3.666000	864.944579
std	7.559024	1.149964	435.712593
min	2.000000	1.000000	95.667400

25%	8.000000	3.000000	490.800000
50%	8.000000	4.000000	867.535000
75%	16.000000	5.000000	1237.910000
max	24.000000	5.000000	1632.720600

```
# Delay analysis
```

```
print("Delay Distribution:")
print(df['delayed'].value_counts())
```

```
print("\nDelay Percentage:")
print(df['delayed'].value_counts(normalize=True) * 100)
```

Delay Distribution:

delayed

no 18331

yes 6669

Name: count, dtype: int64

Delay Percentage:

delayed

no 73.324

yes 26.676

Name: proportion, dtype: float64

```
# Delay analysis by weather condition
```

```
weather_delay = pd.crosstab(
    df['weather_condition'],
    df['delayed'],
    normalize='index'
) * 100
```

```
print(weather_delay)
```

delayed	no	yes
weather_condition		
clear	82.565470	17.434530
cold	83.982684	16.017316
foggy	69.684759	30.315241
hot	82.881356	17.118644
rainy	62.646847	37.353153
stormy	58.551691	41.448309

```
# Delay analysis by vehicle type
```

```
vehicle_delay = pd.crosstab(
    df['vehicle_type'],
    df['delayed'],
    normalize='index'
) * 100
```

```
print(vehicle_delay)
```

	no	yes
delayed		
vehicle_type		
bike	72.956731	27.043269
ev bike	73.565671	26.434329
ev van	73.372206	26.627794
scooter	73.862003	26.137997
truck	72.955368	27.044632
van	73.226654	26.773346

```
# Correlation between distance and delivery time
correlation = df['distance_km'].corr(df['delivery_time_hours'])
print("Correlation between Distance and Delivery Time:", correlation)
```

Correlation between Distance and Delivery Time: 0.6858833649122641

```
# Correlation between distance and delivery cost
correlation = df['distance_km'].corr(df['delivery_cost'])
print("Correlation between Distance and Delivery Cost:", correlation)
```

Correlation between Distance and Delivery Cost: 0.9907716907515611

```
# Correlation between package weight and delivery cost
correlation = df['package_weight_kg'].corr(df['delivery_cost'])
print("Correlation between Package Weight and Delivery Cost:", correlation)
```

Correlation between Package Weight and Delivery Cost: 0.09947871252003065

```
# Delivery status distribution
print(df['delivery_status'].value_counts())
print("\nDelivery Status Percentage:")
print(df['delivery_status'].value_counts(normalize=True) * 100)
```

```
delivery_status
delivered    18331
delayed      5341
failed       1328
Name: count, dtype: int64

Delivery Status Percentage:
delivery_status
delivered    73.324
delayed     21.364
failed       5.312
Name: proportion, dtype: float64
```

```
# Delay rate by delivery partner
```

```

partner_delay = pd.crosstab(
    df['delivery_partner'],
    df['delayed'],
    normalize='index'
) * 100

print(partner_delay)

```

delayed	no	yes
delivery_partner		
amazon logistics	72.777573	27.222427
blue dart	73.052180	26.947820
delhivery	75.197416	24.802584
dhl	73.661670	26.338330
ecom express	73.365173	26.634827
ekart	72.581221	27.418779
fedex	74.840312	25.159688
shadowfax	72.697368	27.302632
xpressbees	71.726822	28.273178

```

# Average delivery time by weather

```

```

weather_time = df.groupby('weather_condition')['delivery_time_hours'].mean().sort_values()
print(weather_time)

```

weather_condition	
stormy	8.807527
rainy	7.804843
foggy	6.486608
hot	4.784504
clear	4.776431
cold	4.773449

Name: delivery_time_hours, dtype: float64

```

# Average delivery time by delivery mode

```

```

mode_time = df.groupby('delivery_mode')['delivery_time_hours'].mean().sort_values()
print(mode_time)

```

delivery_mode	
two day	6.278642
same day	6.244784
standard	6.237472
express	6.230868

Name: delivery_time_hours, dtype: float64

```

# Delay rate by region

```

```

region_delay = pd.crosstab(
    df['region'],
    df['delayed'],
    normalize='index'
)

```

```
) * 100
```

```
print(region_delay)
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('/content/Delivery_Logistics.csv')

# Fix time columns
df['delivery_time_hours'] = pd.to_datetime(
    df['delivery_time_hours']
).dt.nanosecond

df['expected_time_hours'] = pd.to_datetime(
    df['expected_time_hours']
).dt.nanosecond

# Fix delivery ID
df['delivery_id'] = df['delivery_id'].round().astype(int)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
```

```

-----
---
FileNotFoundError                                Traceback (most recent call
last)
/tmp/ipykernel_4008/1476723688.py in <cell line: 0>()
      3
      4 # Load dataset
----> 5 df = pd.read_csv('/content/Delivery_Logistics.csv')
      6
      7 # Fix time columns

----- 4 frames -----
/usr/local/lib/python3.13/dist-packages/pandas/io/common.py in
get_handle(path_or_buf, mode, encoding, compression, memory_map,
is_text, errors, storage_options)
    871         if ioargs.encoding and "b" not in ioargs.mode:
    872             # Encoding
--> 873             handle = open(
    874                 handle,
    875                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory:
'/content/Delivery_Logistics.csv'

```

```

import os

print(os.listdir('/content'))

```

[Show hidden output](#)

```

from google.colab import drive
drive.mount('/content/drive')

```

```

import os

print(os.listdir('/content'))

```

```
['.config', 'archive (1).zip', 'sample_data']
```

```
import zipfile
import os

zip_path = '/content/archive (1).zip'

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    print(zip_ref.namelist())
```

```
['Delivery_Logistics.csv']
```

```
import zipfile

with zipfile.ZipFile('/content/archive (1).zip', 'r') as zip_ref:
    zip_ref.extractall('/content')

print("CSV extracted successfully!")
```

```
CSV extracted successfully!
```

```
import pandas as pd

df = pd.read_csv('/content/Delivery_Logistics.csv')

print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (25000, 15)
```

```
# Clean time columns

df['delivery_time_hours'] = pd.to_datetime(
    df['delivery_time_hours']
).dt.nanosecond

df['expected_time_hours'] = pd.to_datetime(
    df['expected_time_hours']
).dt.nanosecond

# Check missing values and duplicates

print("Missing values:")
print(df.isnull().sum().sum())

print("\nDuplicate rows:", df.duplicated().sum())

print("\nCleaning completed!")
```

```
Missing values:
0
```

```
Duplicate rows: 0
```

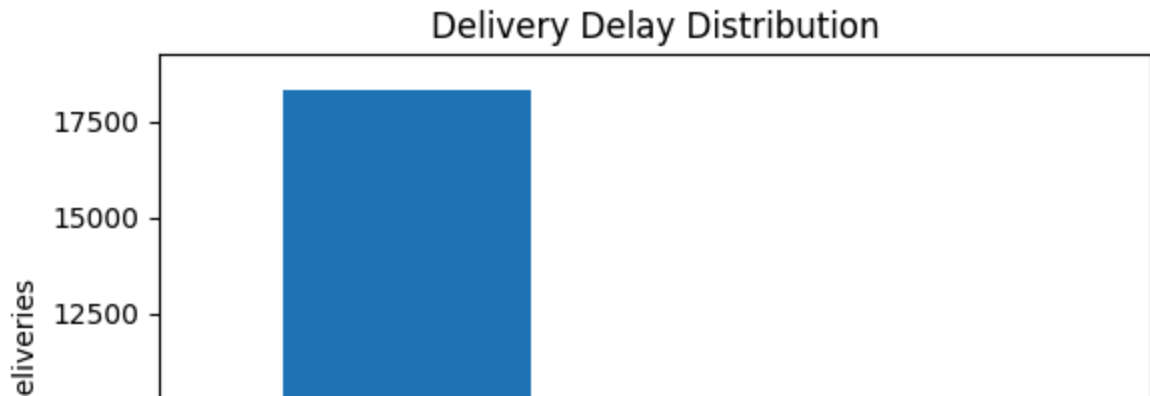
```
Cleaning completed!
```

```
# Delay Distribution Chart

df['delayed'].value_counts().plot(kind='bar')

plt.title('Delivery Delay Distribution')
plt.xlabel('Delayed')
plt.ylabel('Number of Deliveries')

plt.show()
```



```
# Outlier detection using IQR

numeric_cols = [
    'distance_km',
    'package_weight_kg',
    'delivery_time_hours',
    'expected_time_hours',
    'delivery_rating',
    'delivery_cost'
]

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower) | (df[col] > upper)]

    print(col, "→ Outliers:", len(outliers))
```

```
distance_km → Outliers: 0
package_weight_kg → Outliers: 0
delivery_time_hours → Outliers: 203
expected_time_hours → Outliers: 0
delivery_rating → Outliers: 0
delivery_cost → Outliers: 0
```

```
from sklearn.preprocessing import MinMaxScaler

numeric_cols = [
    'distance_km',
```

```
        'package_weight_kg',
        'delivery_time_hours',
        'expected_time_hours',
        'delivery_rating',
        'delivery_cost'
    ]

    scaler = MinMaxScaler()

    df_scaled = df.copy()
    df_scaled[numeric_cols] = scaler.fit_transform(df[numeric_cols])

    print("Normalization completed!")
    print(df_scaled[numeric_cols].head())
```